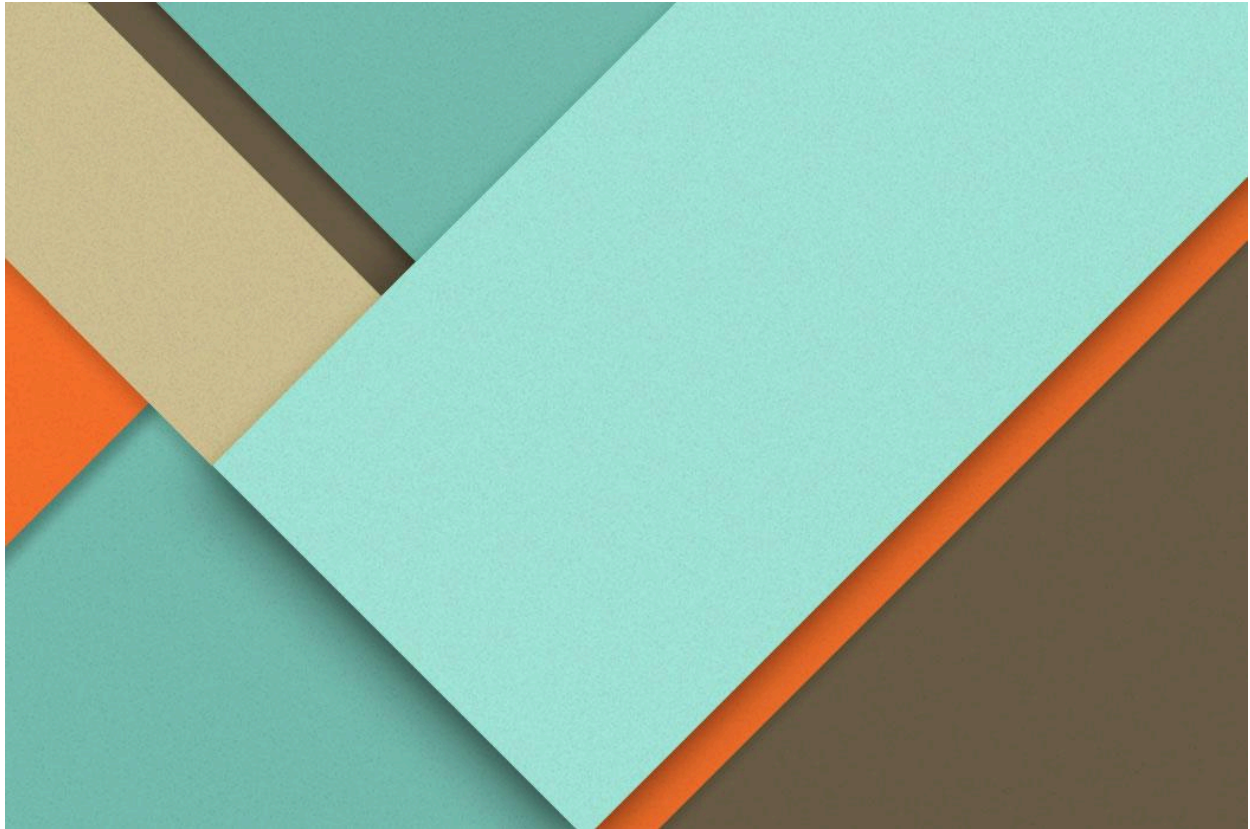




Motores de Juego 2D y 3D

13/04/2026

—

José Manuel Sánchez Rosal

2º DAM

IES Antonio Gala

1400 Palma del Río (Córdoba)

Índice

Índice.....	1
1. Introducción.....	2
1.1 ¿Qué es un motor de videojuegos?.....	2
1.2. Diferencia entre juego 2D y 3D.....	2
2. Análisis de Motores.....	3
2.1. Unity.....	3
2.2 Godot.....	4
libGDX.....	4
2.3. Valoración Crítica y Comparativa.....	5
3. Entornos de Desarrollo.....	5
3.1. Unity.....	5
3.2. Godot.....	6
3.3. libGDX.....	6
4. Componentes de un Motor.....	7
4.1. Motor Gráfico (Renderer).....	7
4.2. Motor Físico.....	7
4.3. Sistema de Entrada (Input System).....	8
4.4. Sistema de Audio.....	8
5. Análisis de Juegos.....	9
5.1. Pruebas de Ejecución y Movimiento.....	9
5.2. Pruebas de Renderizado (Gestión de Profundidad 2D).....	10
5.3. Pruebas de Colisiones y Cajas de Impacto.....	11
5.4. Cambio de Parámetros Dinámicos.....	12
6. Arquitectura del Juego.....	12
6.1. Organización del código.....	12
6.2. Gestión de escenas.....	13
6.3. Objetos principales.....	13
7. Bloques Funcionales.....	14
8. Renderizado.....	16
9. Escena Gráfica.....	16
10. Errores y Depuración.....	17
11. Conclusiones.....	18

1. Introducción

1.1 ¿Qué es un motor de videojuegos?

Un motor de videojuegos (o *game engine*) es un entorno de trabajo o *framework* de software diseñado específicamente para facilitar y agilizar la creación de videojuegos. Su propósito principal es proporcionar un conjunto de herramientas y sistemas base (como el renderizado de gráficos, la simulación de físicas, la detección de colisiones y la gestión de audio) para que los desarrolladores no tengan que programar todo desde cero.

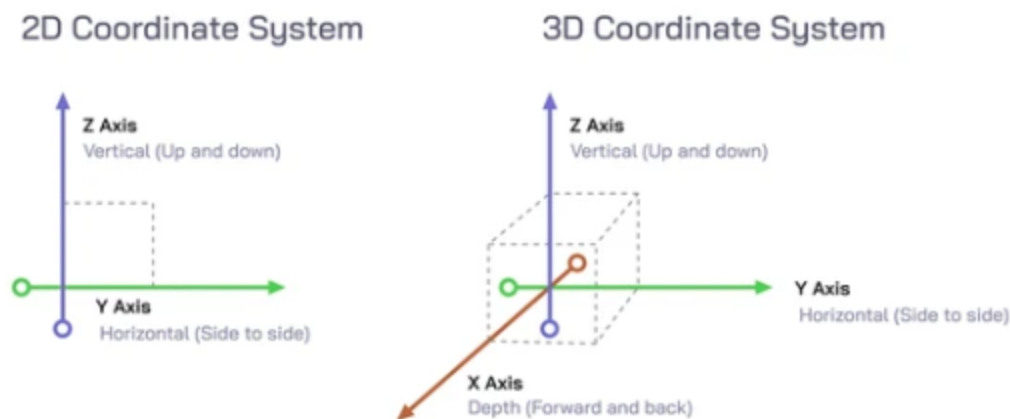
Sin un motor, para crear un juego habría que interactuar directamente con el hardware del dispositivo para indicarle cómo dibujar cada píxel en la pantalla o cómo calcular matemáticamente la gravedad. El motor abstrae toda esa complejidad técnica, permitiendo a los creadores centrarse en lo que realmente importa: el diseño, la lógica, las mecánicas y el arte del juego.

1.2. Diferencia entre juego 2D y 3D

La diferencia fundamental entre los juegos 2D y 3D radica en el sistema de coordenadas que utilizan para representar su mundo y en la naturaleza de sus elementos gráficos:

- **Juegos 2D (Bidimensionales):** Se basan en un sistema de dos ejes cartesianos: X (horizontal) e Y (vertical). No existe una profundidad matemática real. Los elementos visuales son *sprites* (imágenes planas) que se dibujan en la pantalla superponiéndose unos a otros mediante un sistema de capas (o *Z-index*). Aunque un juego pueda tener personajes que caminan "hacia el fondo" del escenario en perspectiva —como ocurre en títulos arcade clásicos de peleas como *Captain Commando*—, el motor sigue tratando los datos como posiciones en un plano de dos dimensiones, engañando al ojo mediante trucos visuales o reduciendo el tamaño del *sprite*.
- **Juegos 3D (Tridimensionales):** Introducen un tercer eje: el eje Z (profundidad). El mundo no es un lienzo plano, sino un espacio volumétrico completo. En lugar de imágenes planas, se utilizan modelos compuestos por mallas poligonales (*meshes*), texturas y materiales. El motor calcula constantemente la geometría espacial, la iluminación y las sombras de estos objetos, y finalmente utiliza una "cámara virtual" para proyectar esa escena tridimensional en la pantalla bidimensional de nuestro monitor.

En resumen, mientras que el 2D organiza imágenes planas en una pizarra, el 3D simula un mundo físico con volumen dentro del cual nos podemos mover libremente:



2. Análisis de Motores

A continuación, se presenta un análisis técnico y comparativo de tres de las herramientas más relevantes en el desarrollo actual de videojuegos: **Unity**, **Godot** y **libGDX**.

2.1. Unity

Tipo: 2D y 3D. Aunque nació como un motor puramente 3D, su módulo 2D ha evolucionado hasta convertirse en un estándar de la industria.

Lenguaje(s): C#.

Características principales: Es un motor comercial muy maduro con un entorno visual completo (editor). Destaca por su sistema de componentes (arquitectura Entity-Component-System) y su gigantesca *Asset Store*. Compila para prácticamente cualquier plataforma actual (móviles, consolas, PC, web).

Ventajas: Comunidad masiva y documentación inagotable.

- Espectacular rendimiento y calidad gráfica en 3D.
- Integración perfecta para monetización y anuncios en dispositivos móviles.

Inconvenientes: Es un motor muy "pesado"; consumir demasiados recursos para proyectos 2D sencillos.

- Su sistema 2D, al estar construido sobre un mundo 3D subyacente (falsificando la profundidad Z), a veces genera complicaciones innecesarias en juegos basados estrictamente en *sprites* (píxeles).

2.2 Godot

Tipo: 2D y 3D. A diferencia de otros motores, Godot tiene un motor de renderizado 2D dedicado que trabaja con coordenadas de píxeles reales, independiente de su motor 3D.

Lenguaje(s): GDScript (lenguaje propio similar a Python), C# y C++.

Características principales: Es un motor de código abierto (licencia MIT). Utiliza una arquitectura innovadora basada en un sistema de "Nodos" y "Escenas" que se pueden anidar infinitamente, lo que facilita mucho la organización del código.

Ventajas: Totalmente gratuito, sin pago de licencias ni *royalties* por ventas.

- Extremadamente ligero (el ejecutable ocupa muy poco y no requiere instalación).
- Su sistema 2D es nativo, lo que lo hace perfecto para desarrollar *beat 'em ups* clásicos de *sprites* sin lidiar con el eje Z 3D.

Inconvenientes: Su capacidad gráfica en 3D, aunque mejora con cada versión, aún no alcanza el nivel fotorrealista de motores como Unity o Unreal.

- Al ser más nuevo, tiene menos *assets* y plugins prefabricados creados por la comunidad.

libGDX

Tipo: 2D y 3D (aunque su uso es inmensamente mayoritario en 2D).

Lenguaje(s): Java (y soporta Kotlin).

Características principales: A diferencia de Unity o Godot, libGDX no es un motor con una interfaz gráfica (editor visual), sino un *framework*. Está basado puramente en código, proporcionando una API unificada que funciona en múltiples plataformas.

Ventajas: Control absoluto a bajo nivel: sabes exactamente qué hace el código en todo momento porque tú estructuras el bucle del juego.

- Rendimiento excepcional en juegos 2D para Android (al estar basado en Java).

Inconvenientes: Curva de aprendizaje muy pronunciada: hay que programar la gestión de la escena, las colisiones y las cámaras "a mano".

- Carece de un editor visual integrado para arrastrar y soltar elementos, lo que ralentiza el diseño de niveles.

2.3. Valoración Crítica y Comparativa

Desde una perspectiva técnica, la elección de la herramienta depende de la naturaleza del proyecto. **Unity** es la opción más segura para la industria comercial y proyectos 3D ambiciosos debido a su robustez, pero peca de ser "matar moscas a cañonazos" en juegos 2D simples. **libGDX** ofrece un control y un aprendizaje a nivel de código sin igual (ideal para afianzar conocimientos de programación en Java), pero los tiempos de desarrollo son notablemente más largos al no disponer de un editor de mapas. Por último, **Godot** se posiciona como la opción más equilibrada y eficiente para el desarrollo en 2D (como juegos tipo arcade retro) gracias a su motor 2D dedicado y a su intuitiva arquitectura de nodos.

3. Entornos de Desarrollo

El entorno de desarrollo es **el espacio de trabajo diario del programador y diseñador**. La elección del motor no solo afecta al rendimiento del juego, sino a la eficiencia del equipo gracias a las herramientas que ofrece su interfaz.

3.1. Unity

- **Instalación:** Se gestiona a través de *Unity Hub*, un gestor externo que permite instalar múltiples versiones del motor simultáneamente. La instalación es pesada y requiere la descarga de varios gigabytes de datos, además de complementos específicos según la plataforma de destino (Android, PC, etc.). Requiere crear una cuenta de usuario obligatoria.
- **Interfaz:** Dispone de un editor visual sumamente completo y profesional. Se divide en paneles clave: la *Jerarquía* (lista de objetos en escena), el *Inspector* (propiedades

del objeto seleccionado), la ventana de *Proyecto* (archivos) y la *Escena/Juego* (visor visual).

- **Facilidad de uso:** Al principio, la inmensa cantidad de botones y opciones puede resultar abrumadora para un principiante. Sin embargo, su flujo de trabajo basado en arrastrar y soltar (*drag & drop*) componentes hace que, una vez superada la curva de aprendizaje inicial, sea muy ágil.
- **Público objetivo:** Estudios profesionales (AA y AAA), desarrolladores independientes con aspiraciones comerciales, y creadores que buscan estandarización y herramientas muy potentes.

3.2. Godot

- **Instalación:** Destaca por su extrema ligereza. No requiere instalación formal; se descarga un archivo comprimido de apenas unos pocos megabytes que contiene un único ejecutable portátil. Se puede llevar y ejecutar desde un pendrive.
- **Interfaz:** Su interfaz es limpia, unificada y muy personalizable (de hecho, el propio editor de Godot está construido con el motor de Godot). Todo gira en torno a su panel de "Árbol de Nodos" y el inspector de propiedades.
- **Facilidad de uso:** Es muy alta. Su filosofía de organizar todo mediante "Nodos" (elementos individuales como un Sprite, una Cámara o un Sonido) que se agrupan en "Escenas", resulta extremadamente intuitiva. Para un juego como *Captain Commando*, colocar los *sprites* y ajustar sus cajas de colisión en su visor 2D es un proceso rápido y directo.
- **Público objetivo:** Desarrolladores *indie*, estudiantes, aficionados (*hobbyists*) y estudios medianos que buscan una herramienta libre de licencias, versátil y especialmente fuerte en entornos 2D.

3.3. libGDX

- **Instalación:** No se instala como un programa tradicional. Se utiliza una herramienta generadora de proyectos (un archivo [.jar](#) de Java) donde configuras el nombre de tu juego y los paquetes. Esta herramienta genera la estructura de carpetas de un proyecto basado en Gradle.
- **Interfaz: Carece de interfaz gráfica o editor visual propio.** El "entorno" es simplemente el IDE de programación (Entorno de Desarrollo Integrado) que elijas, como IntelliJ IDEA, Eclipse o Android Studio.
- **Facilidad de uso:** Muy baja para perfiles visuales (artistas o diseñadores), pero atractiva para programadores puros. Al no haber un visor donde arrastrar objetos,

crear un nivel de *Captain Commando* en libGDX implica escribir código que diga: **dibujar sprite en coordenadas X: 100, Y: 50**. Todo es código.

- **Público objetivo:** Programadores estrictos, estudiantes de ingeniería de software que desean aprender cómo funciona la arquitectura interna de un videojuego sin la "magia" de un editor visual, y desarrolladores muy enfocados en el ecosistema Java/Android.

4. Componentes de un Motor

Un motor de videojuegos no es un bloque de código monolítico, sino un conjunto de sistemas o subsistemas especializados que trabajan en paralelo y se comunican entre sí 60 veces por segundo (o más). Los componentes fundamentales que forman el núcleo de cualquier motor son:

4.1. Motor Gráfico (Renderer)

Es el subsistema encargado de calcular y enviar a la tarjeta gráfica (GPU) la información necesaria para dibujar la escena en la pantalla. Su objetivo es generar cada fotograma (*frame*) lo más rápido posible.

- **En 3D:** Se encarga de procesar mallas poligonales (*meshes*), aplicarles texturas, calcular cómo la luz rebota en los materiales y generar sombras y reflejos en tiempo real mediante pequeños programas llamados *shaders*.
- **En 2D (como en *Captain Commando*):** El motor gráfico se centra en el *Sprite Rendering*. Dibuja imágenes planas en la pantalla, gestionando el orden de dibujado (*Z-index* o capas) para que el personaje se vea "delante" del fondo pero "detrás" de un enemigo si lo cruza, gestionando además las animaciones mediante el cambio rápido de *sprites* (hojas de *sprites* o *spritesheets*).

4.2. Motor Físico

Es el componente que simula las leyes de la física dentro del mundo virtual, garantizando que los objetos interactúen de manera creíble.

- **Cinemática y Dinámica:** Aplica fuerzas matemáticas a los objetos (llamados "cuerpos rígidos" o *RigidBodies*), calculando la gravedad, la fricción, la masa y la inercia.

- **Detección de Colisiones:** Es su función más crítica. En lugar de calcular el choque sobre el modelo visual complejo, utiliza formas geométricas invisibles y simplificadas llamadas *Colliders* (cajas, esferas o cápsulas). En un juego *beat 'em up* 2D, el motor físico detecta cuándo la "caja de ataque" (*hitbox*) del puño del Capitán Commando se solapa con la "caja de daño" (*hurtbox*) del enemigo, registrando el impacto sin necesidad de simular físicas complejas como la gravedad realista.

4.3. Sistema de Entrada (Input System)

Es el puente de comunicación entre el jugador y el juego. Se encarga de capturar, procesar y traducir las señales de los periféricos físicos (teclado, ratón, mando, pantalla táctil) en acciones lógicas dentro del software.

- **Abstracción de controles:** Los motores modernos no programan "si se pulsa la barra espaciadora, salta". En su lugar, utilizan mapas de acciones (*Action Mapping*). Se programa la acción "Saltar" y el sistema de entrada permite asignar a esa acción el botón 'A' del mando, la barra espaciadora o tocar la pantalla, facilitando enormemente que el juego sea multiplataforma.

4.4. Sistema de Audio

Gestiona la carga, decodificación y reproducción de todos los sonidos del juego, desde la banda sonora hasta los efectos especiales (SFX).

- **Mezcla y Canales:** Permite agrupar audios (canal de música, canal de efectos, canal de voces) para que el jugador pueda ajustar sus volúmenes de forma independiente.
- **Audio Espacial:** En juegos 3D, el motor de audio modifica dinámicamente el sonido según la distancia y la posición (paneo izquierdo/derecho) de la fuente respecto a la "oreja virtual" del jugador (normalmente asociada a la cámara). En un entorno 2D, gestiona principalmente el volumen base, el tono (*pitch*) para dar variedad a sonidos repetitivos (como los pasos) y la reproducción instantánea de los efectos de impacto.

En conclusión, estos cuatro componentes no actúan de forma aislada, sino como un engranaje perfecto orquestado por el bucle principal del motor. Mientras el sistema de entrada capta la acción del jugador, el motor físico calcula la interacción, el gráfico la dibuja y el de audio le da vida sonora. Esta abstracción y trabajo en equipo interno es lo que verdaderamente distingue a un motor de videojuegos de una simple librería de programación. Entender esta sinergia es vital para cualquier desarrollador, ya que permite optimizar el rendimiento y diseñar mecánicas eficientes.

5. Análisis de Juegos

Para iniciar esta fase práctica, el primer paso consistió en la localización y puesta en marcha del entorno de trabajo. A diferencia de otros motores comerciales, **Godot** destaca por su extrema ligereza y su filosofía de "descargar y ejecutar". Tras realizar una búsqueda rápida en su portal oficial, se procedió a la descarga de un archivo comprimido de apenas unos pocos megabytes.

El proceso de "instalación" es, en realidad, un simple **desempaquetado**: el motor no requiere una instalación formal en el sistema, ya que consiste en un único ejecutable portátil que puede llevarse y ejecutarse incluso desde un pendrive. Esta naturaleza portable facilitó una configuración instantánea del entorno para comenzar con las pruebas de arquitectura.

 Godot_v4.6.2-stable_win64.exe	13/04/2026 12:53	Aplicación	168.349 KB
 Godot_v4.6.2-stable_win64_console.exe	13/04/2026 12:53	Aplicación	194 KB

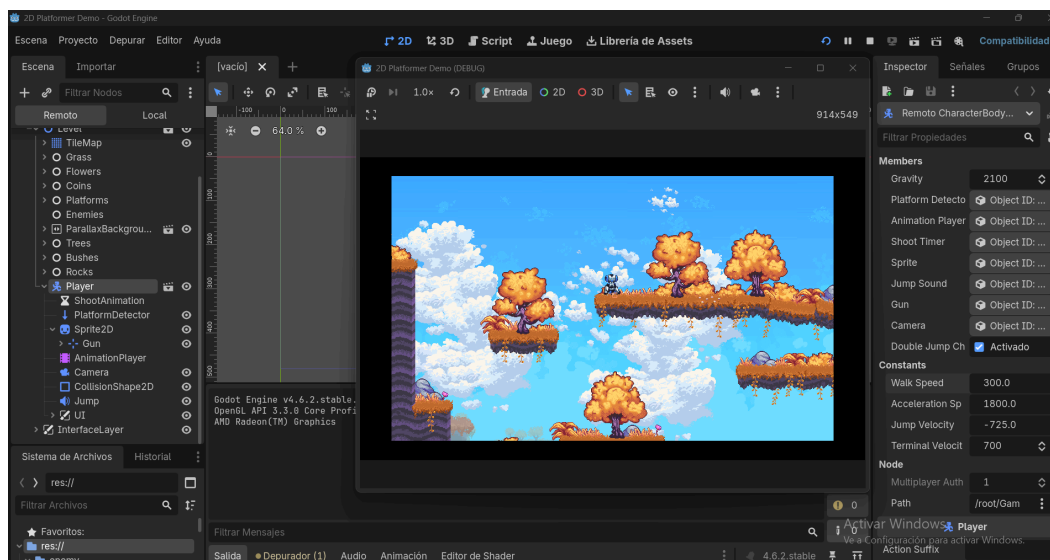
Aunque el marco teórico de este análisis toma como referencia principal el clásico **Captain Commando**, por motivos de accesibilidad al código fuente y para poder realizar pruebas reales de arquitectura y manipulación de parámetros, este estudio se apoya en una plantilla de desarrollo 2D (*2D Platformer*) ejecutada en el motor **Godot**. Esta elección nos permite simular el comportamiento base del título original y observar con gran claridad cómo interactúan los sistemas bidimensionales, la gestión de sprites y las físicas en un entorno controlado.

Se han llevado a cabo las siguientes pruebas técnicas en el motor:

5.1. Pruebas de Ejecución y Movimiento

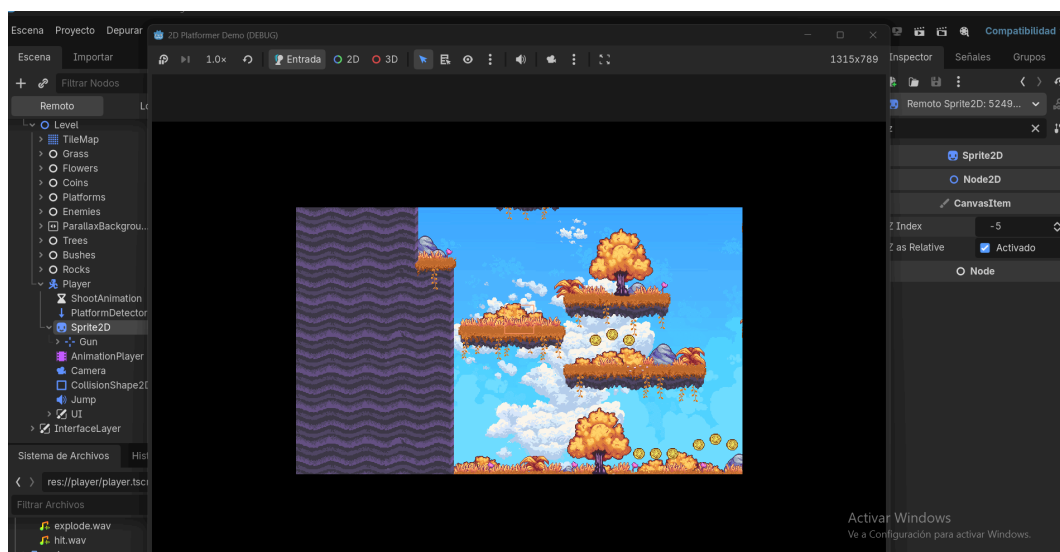
Se ha analizado el desplazamiento del personaje principal a lo largo de los ejes X e Y. Se ha comprobado cómo el motor actualiza la posición del *sprite* en cada fotograma mediante un vector de dirección, respondiendo de forma fluida a los comandos de entrada simulados.

- En la imagen inferior se observa la escena en ejecución junto al inspector de propiedades, donde se puede visualizar el código o los parámetros que controlan la velocidad de movimiento horizontal y vertical:



5.2. Pruebas de Renderizado (Gestión de Profundidad 2D)

Dado que *Captain Commando* carece de un eje Z real, se ha probado el sistema de dibujado por capas (Z-index o *Sorting Layers*). Se modificó en tiempo real el orden de renderizado para comprobar cómo el motor dibuja el fondo primero y los personajes encima, generando la falsa sensación de profundidad al moverse por el escenario. Al modificar el valor del eje "Z index a" -5, el personaje desaparece:



5.3. Pruebas de Colisiones y Cajas de Impacto

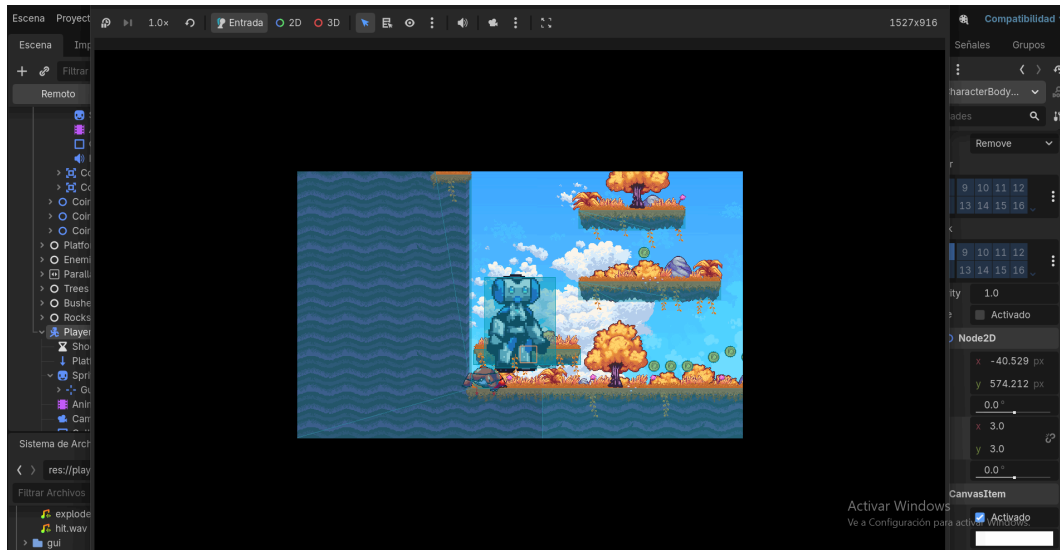
En un *beat 'em up*, la detección de impactos es crítica. Se activó la vista de depuración (*Debug Draw*) para revelar la geometría invisible. Se probó a cambiar el tamaño de las cajas durante una animación de ataque para observar cómo el motor físico registra la intersección matemática.

- Aquí se aprecia la representación lógica de las colisiones. Se muestran las *hitboxes* (cajas de ataque, como el puño del personaje) intersecando con las *hurtboxes* (cajas de vulnerabilidad) del enemigo, lo que dispara el evento de daño sin requerir físicas complejas.



5.4. Cambio de Parámetros Dinámicos

Se realizó una prueba alterando las propiedades espaciales del objeto en caliente. Mientras la escena estaba en ejecución, se modificó el valor de la escala (Scale) del personaje desde el panel del motor para comprobar la respuesta de renderizado inmediata sin necesidad de recompilar la ejecución:



6. Arquitectura del Juego

Para este análisis, se detalla la arquitectura subyacente del prototipo 2D estudiado. La arquitectura se aleja de los bucles monolíticos de programación tradicional para abrazar un modelo jerárquico estructurado mediante nodos y componentes:

6.1. Organización del código

El código no se encuentra centralizado en un único archivo de control maestro, sino que está altamente modularizado. El motor utiliza un sistema de "Scripts adjuntos" (en este caso, mediante el lenguaje GDScript). Cada pieza lógica del juego tiene su propio archivo de código asociado directamente al elemento que controla. Por ejemplo, la lógica de la gravedad, el salto y el movimiento reside exclusivamente en un script llamado `player.gd`, el cual está acoplado únicamente al objeto del jugador. Esto favorece la encapsulación y

facilita la depuración; si un enemigo no patrulla correctamente, el error se aísla en su propio script sin afectar al resto del sistema.

6.2. Gestión de escenas

La arquitectura visual y lógica se fundamenta en un patrón de "Árbol de Nodos" (*Node Tree*). En este paradigma, cualquier elemento es un nodo, y una agrupación de nodos guardada en disco conforma una "Escena". La gestión es puramente jerárquica y permite un anidamiento infinito (instanciación). En el juego analizado existe una escena principal global (**Level** o Nivel), que actúa como contenedor raíz. Dentro de esta escena principal, se "instancian" o inyectan otras sub-escenas completamente independientes y prefabricadas, como la escena del **Player** (Jugador) o las escenas de los **Enemies** (Enemigos). Esta composición modular permite construir mundos complejos ensamblando piezas reutilizables.

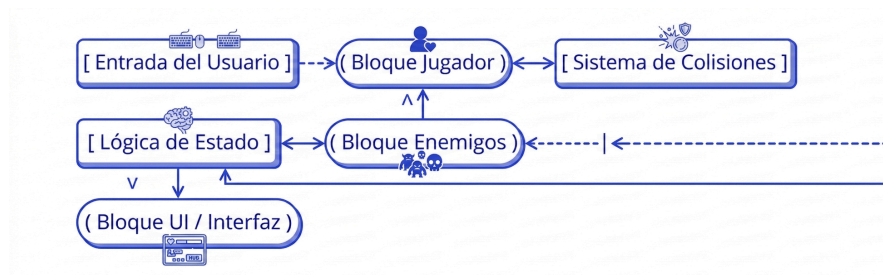
6.3. Objetos principales

Dentro de esta jerarquía modular, los objetos clave que sustentan la jugabilidad se dividen según su naturaleza física y de control:

- **Level / TileMap (Entorno Estático):** Es el objeto contenedor. Se encarga de dibujar el mapa del nivel mediante una cuadrícula de imágenes (*TileMap*) y aplica físicas estáticas (*StaticBody2D*) de colisión al suelo y las paredes para evitar que los actores caigan al vacío. Gestiona de forma pasiva los fondos (*Parallax*).
- **Player (Cuerpo Cinemático):** Es el actor principal (*CharacterBody2D*). Se trata de un objeto dinámico controlado por el sistema de entrada del usuario. Actúa como un contenedor padre que agrupa a sus componentes hijos esenciales: un *Sprite2D* (para su representación gráfica en pantalla) y un *CollisionShape2D* (su caja de impacto física invisible).
- **Enemy / Items (Actores Reactivos):** Entidades dispersas por la escena gestionadas por inteligencia artificial básica o detectores espaciales. Muchos de ellos operan mediante nodos de área (*Area2D*), cuya función arquitectónica principal es escuchar el motor físico para detectar si otros cuerpos (como la caja de colisión del jugador) entran en su espacio, disparando así eventos lógicos como recibir daño o recolectar una moneda.

7. Bloques Funcionales

El funcionamiento del juego se divide en bloques lógicos independientes que se comunican entre sí. A continuación se presenta el diagrama de arquitectura funcional y la descripción de cada bloque:



- **Jugador:** Es el bloque central interactivo. Recibe las señales del *Input System* (teclado/mando) y las traduce en vectores de movimiento. Gestiona sus propios estados lógicos (saltando, corriendo, atacando) y actualiza su animación correspondiente.
- **Enemigos:** Son bloques autónomos regidos por máquinas de estado simples o scripts de patrulla. Tienen parámetros fijos (velocidad, daño) y reaccionan a los límites del mapa o a la proximidad del jugador para cambiar su comportamiento (de patrullar a atacar).
- **UI (Interfaz de Usuario):** Es un bloque funcional pasivo que se encarga de "escuchar" las variables del juego (como la vida del jugador o la puntuación). Arquitectónicamente, se dibuja en una capa independiente (*CanvasLayer*) anclada a la cámara, de forma que el contador de monedas no se quede atrás cuando el personaje avanza por el nivel.
- **Sistema de colisiones:** Es el árbitro invisible del juego. En lugar de procesar los píxeles de los dibujos, el motor calcula intersecciones entre polígonos simples asignados a cada actor. Godot utiliza un sistema de "Capas y Máscaras" (*Layers & Masks*) para optimizar este bloque: define matemáticamente quién puede chocar con quién (ej: el jugador choca con el suelo, pero las nubes del fondo no interactúan con nada).
- **Lógica de Estado (El Controlador):** Actúa como el núcleo de datos del nivel. Sincroniza el comportamiento del **Bloque Enemigos** y, al mismo tiempo, envía información procesada al **Bloque UI**. Es el encargado de decidir, por ejemplo, cuándo una colisión entre el jugador y un enemigo debe restar una vida en la interfaz o cambiar la música de fondo.

Evidencia de Pruebas: Para comprobar la comunicación entre el Bloque Jugador, el Bloque Enemigos y el Sistema de Colisiones, se habilitó la depuración visual del motor gráfico.



Esta captura evidencia de forma muy clara la separación entre la representación visual (*sprites*) y la representación lógica matemática. Se puede observar cómo el motor no calcula los choques píxel a píxel, sino que simplifica la carga de procesamiento utilizando formas geométricas básicas adaptadas a cada entidad:

- **El jugador (centro-abajo) y las plataformas** utilizan cajas de colisión rectangulares estrictas (*RectangleShape2D*) para calcular el soporte y la gravedad de forma precisa.
- **Los enemigos (esquina inferior izquierda y superior derecha)** utilizan formas circulares o de cápsula (*CircleShape2D*), lo que facilita el cálculo de deslizamiento si chocan contra esquinas o bordes.
- **Los elementos decorativos** (nubes, árboles de fondo) carecen por completo de estas formas celestes, demostrando que el motor físico los ignora para optimizar el rendimiento del renderizado.

8. Renderizado

El renderizado es el proceso mediante el cual el motor traduce la información lógica y matemática en los píxeles visibles que el usuario observa en pantalla. Este proceso se realiza cíclicamente docenas de veces por segundo para generar la sensación de movimiento fluido.

- **¿Qué es el renderizado?:** Es el subsistema encargado de enviar a la tarjeta gráfica la información necesaria para dibujar la escena.
- **Diferencias 2D vs 3D:**
 - **En 3D:** Se procesan mallas poligonales (*meshes*), texturas y materiales, calculando cómo la luz rebota y genera sombras en un espacio volumétrico.
 - **En 2D:** El motor se centra en el *Sprite Rendering*, dibujando imágenes planas sobre un lienzo bidimensional siguiendo un orden de superposición.
- **Cómo se dibuja la escena:** Se utiliza el sistema de **Z-index** o capas. El motor dibuja los elementos de atrás hacia adelante: primero los fondos, luego los obstáculos y finalmente los personajes y la interfaz.

Evidencia de Pruebas: Para comprobar este sistema, se alteró el valor del **Z-index** del jugador a **-5**. Como resultado, el renderizador dibujó al personaje por detrás del escenario, ocultándolo visualmente pero manteniendo su existencia lógica en la escena.

9. Escena Gráfica

La escena gráfica es la estructura jerárquica de datos que organiza todos los elementos del mundo virtual dentro del motor.

- **Representación lógica (datos):** Se refiere a la información técnica de cada objeto almacenada en la memoria, como su nombre, tipo de nodo y variables personalizadas (ej: velocidad, vida o gravedad). Es la "mente" del objeto que dicta su comportamiento.
- **Representación espacial (posición, cámara...):** Es la interpretación de esos datos en el plano cartesiano.
 - **Posición:** Define las coordenadas (X , Y) de cada objeto en el espacio.
 - **Escala y Rotación:** Determinan el tamaño y la orientación del objeto en la escena.
 - **Cámara:** Es el visor que decide qué parte de la escena gráfica se proyecta finalmente en la pantalla bidimensional del monitor.

Evidencia de Pruebas: Se realizó una manipulación de la **Escala (Scale)** en caliente. Al cambiar el valor lógico de \$1.0\$ a \$3.0\$ en el inspector, la representación espacial del personaje se transformó instantáneamente en la pantalla, multiplicando su tamaño sin necesidad de alterar el código fuente o reiniciar la ejecución.

10. Errores y Depuración

Durante el desarrollo de las pruebas técnicas en el motor Godot, se identificaron diversos comportamientos que requieren el uso de herramientas de depuración para garantizar la estabilidad del juego.

- **Problemas detectados:**

- **Solapamiento de capas (Z-Fighting/Layering):** Al inicio de las pruebas, el personaje podía quedar oculto tras elementos del escenario. Mediante la consola de depuración y el inspector, se identificó que el valor del **Z-Index** era erróneo.
- **Inconsistencia en colisiones:** Se detectaron colisiones fantasma o falta de respuesta ante impactos. Gracias al modo de depuración visual (*Visible Collision Shapes*), se pudo observar que algunas cajas de colisión no estaban alineadas con el *sprite* o tenían un tamaño inadecuado para la mecánica del juego.
- **Salida de límites (Out of Bounds):** Al manipular la velocidad y la escala en caliente, el actor principal podía atravesar paredes o caer al vacío infinito si las fuerzas físicas superaban los límites de los *colliders*.

- **Soluciones implementadas y herramientas:**

- **Uso del panel Remoto:** Esta herramienta fue vital para corregir errores de escala y velocidad en tiempo real sin reiniciar el juego, permitiendo encontrar el equilibrio exacto para las mecánicas.
- **Normalización de Z-Index:** Se estableció una jerarquía lógica de renderizado donde el fondo siempre ocupa valores negativos y los actores valores positivos, evitando que el jugador "desaparezca".
- **Ajuste de Collision Masks:** Se configuraron las capas y máscaras físicas para asegurar que el motor solo procese los impactos necesarios (ej. Jugador contra Enemigo), ahorrando recursos de CPU.

11. Conclusiones

- **Motor preferido:** Tras analizar Unity, libGDX y Godot, este último se posiciona como la herramienta más eficiente para proyectos 2D. Su arquitectura de nodos es mucho más intuitiva que el sistema puramente basado en código de libGDX, y resulta mucho más ligero y rápido de ejecutar que Unity para desarrollos *indie*.
- **Dificultades encontradas:** La mayor dificultad fue comprender la transición de la representación lógica a la espacial, especialmente al gestionar el orden de renderizado y las cajas de colisión para que el comportamiento físico fuera creíble.
- **Qué has aprendido:** Este trabajo ha servido para entender que un motor de videojuegos no es solo un programa para dibujar, sino un ecosistema complejo donde el motor gráfico, el físico y el sistema de entrada deben trabajar en perfecta sinergia para que la experiencia de juego sea funcional.